

1. Introduzione

1. Introduzione



**** — utile per la stampa o la lettura offline.

Benvenuto/a a bordo!

Ciao, e benvenuto/a in azienda!

Se stai leggendo questa pagina, probabilmente ti sei appena seduto/a alla tua scrivania (fisica o virtuale) con un mucchio di acronimi nuovi in testa — Scrum, Sprint, DevOps, CI/CD, board, backlog — e la sensazione di dover imparare un'intera lingua straniera in poche settimane.

Respira: è normale, ed è esattamente per questo che esiste questo corso.

Sei stato/a assunto/a come **Junior Project Manager** e, nell'arco dei prossimi 2-3 mesi, affiancherai (e poi gradualmente sostituirai) una collega che oggi lavora come **Scrum Master / Project Manager** su un team che si occupa di una **piattaforma DevOps a supporto dello sviluppo software**. Non sai cosa significhi tutto questo? Perfetto, è il punto di partenza giusto: questo corso è pensato apposta per chi, come te, arriva da un percorso di studi gestionale (Ingegneria Gestionale) e parte da zero dal punto di vista tecnico e informatico.

Non ti verrà chiesto di scrivere codice o di diventare uno sviluppatore. Ti verrà chiesto di **capire come funziona il mondo in cui il tuo team lavora**, per poterlo organizzare, facilitare e rappresentare al meglio — verso il team stesso, verso i responsabili e verso il cliente.

Come è organizzato questo corso

Il corso è diviso in **19 sezioni numerate**, pensate per essere seguite **in ordine progressivo**: ogni sezione si appoggia sui concetti spiegati in quella precedente, un po' come i mattoncini di un Lego — non puoi costruire il tetto se non hai ancora messo le fondamenta.

Le sezioni si possono raggruppare in quattro grandi blocchi:

1. **Fondamenta tecniche** (sezioni 2-4): cosa è un computer, come “nasce” un software, cos'è Git/GitHub. Ti servono per capire il linguaggio di base che sviluppatori e tecnici usano ogni giorno.
2. **Metodologie di lavoro** (sezioni 5-8): Agile, Scrum, Kanban e Project Management. Sono i “metodi di organizzazione del lavoro” — qui il tuo background gestionale ti aiuterà moltissimo, perché sono concetti di organizzazione applicati al software.
3. **Mondo DevOps** (sezioni 9-14): DevOps, Azure DevOps, CI/CD, architetture software, cloud e sicurezza. È il cuore tecnico del corso, quello più legato al ruolo che andrai a occupare.
4. **Strumenti di supporto e consultazione** (sezioni 15-19): ambienti di sviluppo, glossario, piano di studio, libri e risorse online. Non sono da “studiare” una volta e basta: sono pensate per essere consultate spesso, come un dizionario o un'agenda.

Esempio pratico: se in una riunione senti dire “la pipeline è rossa, il rilascio è bloccato”, non serve che tu capisca subito cosa significhi nel dettaglio. Ti basta pensare “questo è un argomento del Blocco C, probabilmente della sezione 11 sul CI/CD” e prendere nota mentale per approfondirlo quando arrivi lì, invece di sentirti perso/a o di dover fingere di aver capito.

Oltre alle 19 sezioni, trovi due strumenti trasversali che ti accompagneranno per tutto il percorso:

- **Il Glossario****** (sezione 16): ogni volta che incontri un termine che non ricordi — sia in questo corso, sia durante una riunione di lavoro — puoi cercarlo lì. Non c'è nulla di male nel non ricordare a memoria cosa significa “backlog” la terza volta che lo senti: è normale, ci vuole ripetizione.
- **Il Piano di studio****** (sezione 17): un programma settimanale su **8 settimane** che ti dice, settimana per settimana, cosa leggere, quanto tempo dedicarci, e con quali attività pratiche verificare di aver capito (ad esempio: “questa settimana osserva una Daily Scrum del team e prendi nota di cosa succede”).

Consiglio pratico: la prima volta che apri il corso, dai una scorsa veloce a tutti i titoli delle 19 sezioni (li trovi anche nel menu laterale). Non devi capire tutto subito: ti basta avere una mappa mentale di “cosa troverò più avanti”, così quando in una sezione si accenna a un concetto che verrà spiegato dopo, saprai che è normale e che arriverà il suo momento.

Il contesto di lavoro: cosa fa il tuo team, in parole semplici

Prima di addentrarci nei dettagli tecnici (che vedremo dalla sezione 2 in poi), proviamo a capire **a grandi linee** cosa significa lavorare su “una piattaforma DevOps a supporto dello sviluppo software”. Niente gergo tecnico per ora: solo un'analogia.

Immagina una grande cucina di un ristorante che deve servire centinaia di piatti ogni giorno, in modo veloce, corretto e senza errori. In quella cucina:

- ci sono **cuochi** che preparano i piatti (gli **sviluppatori**, che scrivono il codice del software);
- ci sono **linee di produzione** che passano il piatto da una stazione all'altra — dalla preparazione, alla cottura, all'impiattamento, al controllo qualità, fino al tavolo del cliente (questo,

nel mondo software, si chiama **pipeline**, e lo vedremo nella sezione 11 sul CI/CD);

- ci sono **strumenti e attrezzature** condivise da tutta la cucina — i forni, i frigoriferi, i banchi di lavoro (nel mondo software, sono i **server**, **gli ambienti di test**, **gli strumenti di collaborazione**: lo vedremo nelle sezioni su DevOps, Azure DevOps e Cloud);
- e c'è un **responsabile di sala/cucina** che si assicura che gli ordini arrivino in tempo, che i cuochi non siano sovraccarichi, che eventuali problemi (un piatto tornato indietro, un ingrediente finito) vengano gestiti e comunicati al cliente. Questo è, in sostanza, il ruolo di **Project Manager / Scrum Master**: non cucini tu, ma fai in modo che la cucina funzioni, che il team lavori bene insieme, e che il cliente sia informato e soddisfatto.

Una **piattaforma DevOps** è l'insieme di strumenti e processi che permettono a questa "cucina" di funzionare in modo automatico, veloce e sicuro: dal momento in cui un cuoco scrive una nuova ricetta (scrive codice), al momento in cui quel piatto arriva davanti al cliente (il software è "in produzione", cioè realmente usato dagli utenti finali), passando per tutti i controlli di qualità intermedi.

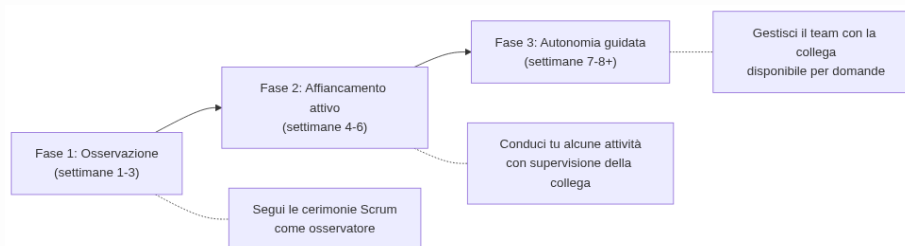
Non preoccuparti se questa descrizione ti sembra ancora vaga: è voluto. Da qui in avanti, sezione dopo sezione, ogni singolo "ingrediente" di questa cucina (Git, Scrum, Kanban, CI/CD, Cloud, Sicurezza...) verrà spiegato nel dettaglio, con esempi concreti tratti dal lavoro quotidiano del team.

Esempio pratico: uno sviluppatore del team completa una nuova funzionalità (es. un nuovo filtro di ricerca nell'applicazione) e la carica su Git (il "quaderno delle ricette", lo vedremo nella sezione 4). Da quel momento, in automatico, una serie di controlli (i "controlli qualità" della pipeline) verificano che il codice non rompa nulla di esistente. Se tutto va bene, la funzionalità arriva prima in un ambiente di test, poi — dopo l'ok del team — in produzione, cioè visibile agli utenti reali della piattaforma. Il tuo ruolo, in questo flusso, non è scrivere codice: è assicurarti che tutti sappiano cosa sta succedendo, che eventuali blocchi (un test che fallisce, un am-

biente non disponibile) vengano comunicati subito, e che il cliente sia aggiornato su quando la funzionalità sarà disponibile.

Il tuo percorso: da Junior Project Manager a Scrum Master

Il tuo percorso di crescita in questi 2-3 mesi seguirà tre fasi, che si sovrappongono gradualmente:



Diagramma

- **Fase 1 - Osservazione:** parteciperai alle riunioni del team (le vedremo nella sezione 6 su Scrum: si chiamano “cerimonie” o “eventi”, come la Daily Scrum o lo Sprint Planning) semplicemente osservando. In questa fase il tuo compito principale è studiare le sezioni di questo corso e collegare quello che leggi a quello che vedi succedere dal vivo.
- **Fase 2 - Affiancamento attivo:** comincerai a condurre tu stesso/a alcune attività — ad esempio facilitare una Daily Scrum, aggiornare la board del progetto, preparare un report per il cliente — mentre la tua collega osserva e ti dà feedback.
- **Fase 3 - Autonomia guidata:** gestirai in autonomia le attività di routine del ruolo, con la tua collega disponibile per domande su casi più complessi, finché il passaggio di consegne non sarà completo.

Questo non è un percorso “accademico” fine a se stesso: è un percorso pensato per portarti, passo dopo passo, a **sostituire operativamente** una persona con un ruolo di responsabilità. Per questo è fondamentale che, oltre a leggere le sezioni, tu **osservi il lavoro reale del team** e faccia domande — tante domande — alla tua collega e al resto del team.

Esempio pratico: nella Fase 1, durante una Daily Scrum, ti limiti ad ascoltare e prendere appunti su chi dice cosa. Nella Fase 2, è la tua collega a chiederti: “oggi la conduci tu, io resto in ascolto e ti do feedback alla fine” — magari dovrai gestire un membro del team che si dilunga troppo su un problema tecnico, riportando la discussione sui binari giusti. Nella Fase 3, sei tu a organizzare la Daily Scrum in autonomia, e la tua collega interviene solo se le chiedi aiuto su un caso particolare (ad esempio, come gestire un conflitto tra due membri del team sulle priorità).

Come affrontare lo studio: qualche consiglio pratico

Prima di iniziare, qualche indicazione su come vivere questo percorso senza scoraggiarti:

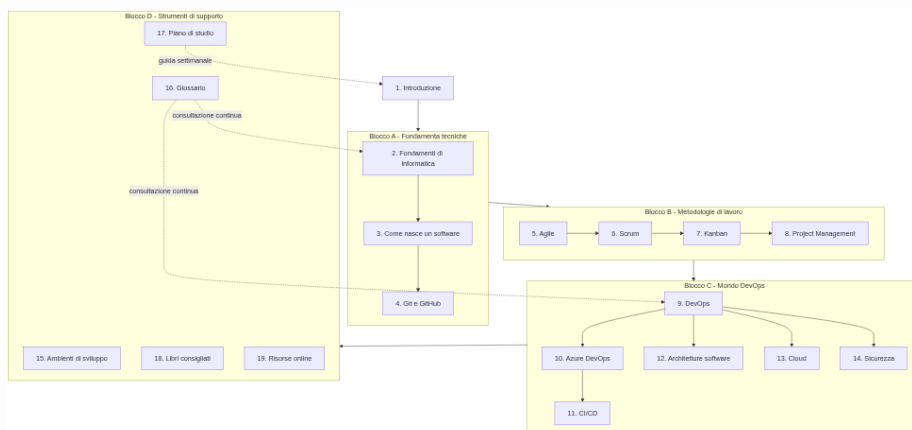
1. **Non devi capire tutto la prima volta.** Concetti come “pipeline CI/CD” o “architettura a microservizi” richiedono più di una lettura per sedimentare. È previsto che tu debba tornare indietro e rileggere: non è un tuo limite, è la normalità quando si impara qualcosa di completamente nuovo.
2. **Fai domande, sempre.** Non esiste una domanda “troppo banale” in questo percorso. Chi lavora ogni giorno con questi strumenti a volte dà per scontati concetti che per te sono nuovi: chiedere è il modo più veloce per colmare quel divario, molto più veloce che aspettare di capire tutto da solo/a leggendo.
3. **Collega la teoria a quello che vedi sul campo.** Ogni volta che in una riunione o in una chat di lavoro senti un termine nuovo, provalo a ricercare nel **Glossario** o pensa a quale sezione del corso lo tratta. Questo doppio binario (teoria + osservazione pratica) è il modo più efficace per imparare in questo contesto.
4. **Prova gli esempi, non limitarti a leggerli.** Dove il corso propone un esempio pratico (una board Kanban, un comando Git, un diagramma di pipeline), se possibile provalo tu stesso/a in un ambiente di prova. Capire leggendo è utile, ma capire “con le mani” resta molto più solido.

5. **Usa il piano di studio come bussola, non come gabbia.** Il **Piano di studio** a 8 settimane è una guida, non un obbligo rigido: se una settimana hai bisogno di più tempo su un argomento (capita spessissimo con i fondamenti di informatica, sezione 2), prenditelo. È molto meglio capire bene una sezione in più tempo che correre e arrivare in fondo con basi fragili.
6. **A fine settimana, fai un check-in con la tua collega o il tuo mentor.** Un breve confronto settimanale ti permette di validare cosa hai capito, chiarire dubbi rimasti in sospeso e ricevere indicazioni su cosa osservare la settimana successiva nel lavoro reale del team.

Esempio pratico: leggi nella sezione 7 cos'è un limite di WIP (Work In Progress) su una board Kanban. Il giorno dopo, durante una riunione, senti la tua collega dire “non possiamo prendere in carico un'altra card, siamo già al limite in ‘In Corso’”. Invece di lasciarlo passare, collegalo subito a quanto letto e magari chiedi: “è questo il limite di WIP di cui si parlava nella sezione 7?”. Questo piccolo gesto — collegare una lettura a una frase sentita dal vivo — è esattamente il “doppio binario” di cui si parla al punto 3.

La mappa del corso

Il diagramma seguente mostra come le 19 sezioni si collegano tra loro a grandi linee. Non è necessario memorizzarlo: serve solo per farti un'idea di dove ti troverai, sezione dopo sezione.



Diagramma

In sintesi: prima impari **il linguaggio di base** del software (Blocco A), poi **il modo in cui i team si organizzano** per costruirlo (Blocco B), poi **gli strumenti e i processi specifici** della piattaforma DevOps che gestirai (Blocco C), e infine hai a disposizione un **kit di strumenti di consultazione** (Blocco D) da usare per tutta la durata del percorso e anche dopo.

Pronto/a per iniziare?

Nella prossima sezione, **2. Fondamenti di informatica**, partiremo dalle basi più elementari: cosa è un computer, come funziona una rete, cosa sono un server e un database. Sono i mattoncini su cui si costruirà tutto il resto del corso, quindi prenditi il tempo che serve.

Buon percorso!

Esercizi pratici

Prima di passare alla sezione 2, prova questi esercizi: non richiedono ancora conoscenze tecniche, servono solo a farti “digerire” la mappa del corso e il contesto in cui lavorerai.

1. **Disegna la mappa del corso a modo tuo.** Su un foglio o in un documento, riscrivi con parole tue (senza copiare) i quattro blocchi (Fondamenta tecniche, Metodologie di lavoro, Mondo DevOps, Strumenti di supporto) e per ciascuno annota una sola parola chiave che ti aiuti a ricordarlo. **Come verificare:** se riesci a spiegare a voce alta, in meno di 90 secondi e senza guardare il foglio, “di cosa parla ciascuno dei quattro blocchi”, l’esercizio è riuscito.
2. **Racconta l’analogia della cucina a qualcuno.** Prova a spiegare a un amico o familiare (anche non del settore) l’analogia della cucina del ristorante usata in questa sezione, sostituendo i termini tecnici con quelli che hai appena imparato (cuochi = sviluppatori, pipeline = linea di produzione, ecc.). **Come**

verificare: se la persona a cui lo racconti riesce a ripeterti, con parole sue, cosa fa un Project Manager/Scrum Master in questa analogia, hai comunicato bene il concetto.

3. **Individua le tue tre fasi nel calendario.** Segna sul tuo calendario (anche approssimativamente) le date di inizio previste delle tre fasi del tuo percorso (Osservazione, Affiancamento attivo, Autonomia guidata) in base a quando sei stato/a assunto/a.
Come verificare: mostra il calendario alla tua collega Scrum Master/PM e chiedile se le date che hai stimato sono realistiche rispetto al carico di lavoro reale del team in quel periodo.
4. **Fai una prima esplorazione del Glossario.** Apri il **Glossario** e scorilo per 5 minuti senza l'ansia di memorizzare nulla: individua 3 termini che hai già sentito nominare in azienda (anche di striscio, in corridoio o in chat) ma che non sapevi spiegare con precisione.
Come verificare: scrivi i 3 termini scelti e una definizione con parole tue in una nota; a fine settimana 1 del piano di studio, riguardala e controlla se la tua definizione è ancora corretta o va corretta.
5. **Identifica chi è chi nel tuo contesto di lavoro.** Chiedi alla tua collega o a un membro del team di aiutarti a fare un elenco veloce (basta un elenco a punti, non serve un documento formale) di chi sono, nel progetto reale su cui lavorerai, gli "sviluppati/cuochi", chi si occupa di "operations" e chi è il referente lato cliente.
Come verificare: se dopo questo esercizio sai dire, senza esitare, a chi rivolgerti per un problema tecnico e a chi per una domanda del cliente, l'esercizio ha funzionato.

Collegamenti

- **2. Fondamenti di informatica** — le basi tecniche da cui parte tutto il resto del corso
- **16. Glossario** — da consultare ogni volta che incontri un termine nuovo
- **17. Piano di studio** — il programma dettagliato delle 8 settimane

- **19. Risorse online** — materiali extra se vuoi approfondire fin da subito

Risorse

- [Atlassian - Agile Coach: cos'è un team Agile](#) — introduzione generale al mindset Agile, utile come primo assaggio prima delle sezioni 5-7
- [GitHub Docs - Introduzione a Git e GitHub](#) — guida ufficiale per chi non ha mai usato Git, utile in anticipo sulla sezione 4
- [Microsoft Learn - Cos'è DevOps](#) — panoramica ufficiale Microsoft sul concetto di DevOps, come anteprima della sezione 9
- [Project Management Institute - What is Project Management](#) — visione generale del ruolo di Project Manager, come richiamo al tuo percorso di studi